Python Cheetsheet

Recent Questions

List of questions

https://leetcode.com/discuss/post/6185127/2024-google-interview-questions-compilat-mjrf/

Interview questions from leetcode discuss

https://leetcode.com/discuss/post/6636134/google-tps-could-only-come-up-brute-forc-sd5j/
https://leetcode.com/discuss/post/6633777/google-l4-phone-screen-by-anonymous_user-gh1z/
https://leetcode.com/discuss/post/6633452/google-phone-screen-by-anonymous_user-4dud/
https://leetcode.com/discuss/post/6633090/google-phone-screen-by-anonymous_user-87p7/
https://leetcode.com/discuss/post/6631018/google-l4-swe3-by-anonymous_user-qyud/
https://leetcode.com/discuss/post/6630660/google-l5-round-2-by-anonymous_user-swe2/
https://leetcode.com/discuss/post/6629122/screening-round-with-google-by-anonymous-im7k/
https://leetcode.com/discuss/post/6628533/google-phone-screen-interview-round-by-a-gxhk/
https://leetcode.com/discuss/post/6626915/google-phone-screen-l4-change-recruiter-ogkke/
https://leetcode.com/discuss/post/6626770/google-l4-telephone-screen-by-anonymous_-rkx9/
https://leetcode.com/discuss/post/6626514/google-l3-tpsmarch-by-bhima2001-x7jv/
https://leetcode.com/discuss/post/6616428/google-l5-round-1-india-by-anonymous_use-qgdd/
https://leetcode.com/discuss/post/6623144/google-phone-screening-round-sde-3-by-an-mz3l/
https://leetcode.com/discuss/post/6623008/google-l3-interview-experience-by-anonym-ybia/
https://leetcode.com/discuss/post/6622332/google-oa-question-3032025-onsite-interv-iiu8/
https://leetcode.com/discuss/post/6618127/google-interview-experience-fresher-swe-tgyxx/
https://leetcode.com/discuss/post/6617221/google-l3-interview-experience-by-anonym-nt48/
https://leetcode.com/discuss/post/6678140/google-l3-optional-coding-assessment-202-st0z/
https://leetcode.com/discuss/post/6676870/google-l4-onsite-1-by-anonymous_user-ehu5/
https://leetcode.com/discuss/post/6676167/google-l4-phone-screening-april-2025-by-e325f/
https://leetcode.com/discuss/post/6676028/google-l4-phone-screening-april-2025-by-dp8zr/
https://leetcode.com/discuss/post/6676008/google-swe-phone-screening-rejected-by-a-bl7s/
https://leetcode.com/discuss/post/6672500/google-mock-test-by-anonymous_user-hed8/

Interview questions from leetcode discuss

https://leetcode.com/discuss/post/6671267/google-onsite-l4-by-anonymous_user-fggd/
https://leetcode.com/discuss/post/6670944/google-phone-screen-l3-verdict-passed-by-wl7a/
https://leetcode.com/discuss/post/6670584/google-onsite-l3-by-anonymous_user-c3w9/
https://leetcode.com/discuss/post/6669726/google-l3-interview-experience-by-anonym-5kfz/
https://leetcode.com/discuss/post/6663236/google-phone-screen-round-by-anonymous_u-prem/
https://leetcode.com/discuss/post/6659499/google-onsite-round-2-by-anonymous_user-u340/
https://leetcode.com/discuss/post/6685598/google-l4-1st-onsite-round-by-anonymous_-jw3q/
https://leetcode.com/discuss/post/6684774/google-l4-interview-experience-usa-by-an-bre1/
https://leetcode.com/discuss/post/6683848/google-onsite-round-1-by-anonymous_user-rj6b/
https://leetcode.com/discuss/post/6683794/google-interview-onsite-by-anonymous_use-kb7l/

https://leetcode.com/discuss/post/6701617/google-onsite-interview-experience-mar-2-h182/
https://leetcode.com/discuss/post/6701549/google-l4-interview-experience-by-anonym-izsb/
https://leetcode.com/discuss/post/6701519/google-l4-interview-by-anonymous_user-kmeq/
https://leetcode.com/discuss/post/6701291/help-google-onsite-interviews-l3-by-anon-5ycm/
https://leetcode.com/discuss/post/6701285/google-screen-round-l4-swe-by-anonymous_-snp5/
https://leetcode.com/discuss/post/6698886/google-onsite-round-1-question-by-anonym-ra0i/
https://leetcode.com/discuss/post/6698775/google-oa-question-by-anonymous_user-6y0k/
https://leetcode.com/discuss/post/6698309/google-screening-round-selected-by-anony-h3n0/
https://leetcode.com/discuss/post/6696473/google-l3-onsite-r1-by-anonymous_user-n8rl/
https://leetcode.com/discuss/post/6696389/google-l3-onsite-r2-by-anonymous_user-bwpk/
https://leetcode.com/discuss/post/6696099/google-l3-onsite-r3-by-anonymous_user-8nma/

先問清楚題目和input range, type，儘量自己定義好問可不可以

給兩個解法，確認可以再繼續寫

加好註解 邊寫邊解釋作法(要當作對方不會python)

dry run

time complexity 分析

Dynamic Programming

Longest Increasing Subsequence (LIS)

```
n = len(nums)
dp = [1 for i in range(n)]
for i in range(len(nums)):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)

return max(dp)
```

```
# Optimal solution with binary search (Patience sort)
end_deck = []
for num in nums:
    idx = bisect.bisect_left(end_deck, num)
    if idx == len(end_deck):
        end_deck.append(num)
    else:
        end_deck[idx] = num

return len(end_deck)
```

Number of LIS

```
# O(N^2) time complexity
n = len(nums)
dp = [1 for i in range(n)] # dp[i] = longest length end at num[i]
count = [1 for in range(n)]
for i in range(n):
    for j in range(i):
        if nums[j] < nums[i]:
            if dp[i] < dp[j] + 1:
                dp[i] = dp[j] + 1
                count[i] = count[j]
            elif dp[i] == dp[j] + 1:
                count[i] += count[j]

max_length = max(dp)
total_count = sum(count[i] for i in range(len(count)) if dp[i] ==
max_length)
```

```
# O(N log N)
class BIT:
    def __init__(self, size):
        self.size = size
        self.lengths = [0] * (size + 1)
        self.counts = [0] * (size+1)
    def update(self, i, length, count):
        while i <= self.size:
            …
            i += i & (-i)
    def query(self, i) -> Tuple[int, int]:
        max_len, count = 0, 0
        while i > 0:
            …
            i -= i & (-i)
sorted_nums = sorted(set(nums))
index = {num: idx + 1 for idx, num in enumerate(sorted_nums)}
bit = BIT(len(index))
```

Number of LIS with Binary Index Tree

```
class BIT:
    def __init__(self, size):
        self.size = size
        self.lengths = [0] * (size + 1)
        self.counts = [0] * (size+1)
    def update(self, i, length, count):
        while i <= self.size:
            if self.lengths[i] < length:
                self.lengths[i] = length
                self.counts[i] = count
            elif self.lengths[i] == length:
                self.counts[i] += count
            i += i & (-i)
    def query(self, i) -> Tuple[int, int]:
        max_len, count = 0, 0
        while i > 0:
            if self.lengths[i] > max_len:
                max_len = self.lengths[i]
                count = self.counts[i]
            elif self.lengths[i] == max_len:
                count += self.counts[i]
            i -= i & (-i)
```

```
# TC = O(N log N): very complicate with binary index tree
def numberOfLIST(nums):
    sorted_nums = sorted(set(nums))
    index = {num: idx + 1 for idx, num in
enumerate(sorted_nums)}
    bit = BIT(len(index))
    for num in nums:
        idx = index[num]
        max_len, count = bit.query(idx-1)
        bit.update(idx, max_len+1, max(count, 1))

    return bit.query(len(index))
```

Longest Common Subsequence (LCS)

```
dp = [[0 for j in range(len(s2)+1)] for i in range(len(s1)+1)]

for i in range(1, len(s1)+1):
    for j in range(1, len(s2) + 1):
        if s1[i-1] == s2[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])

return dp[len(s1)-1][len(s2)-1]
```

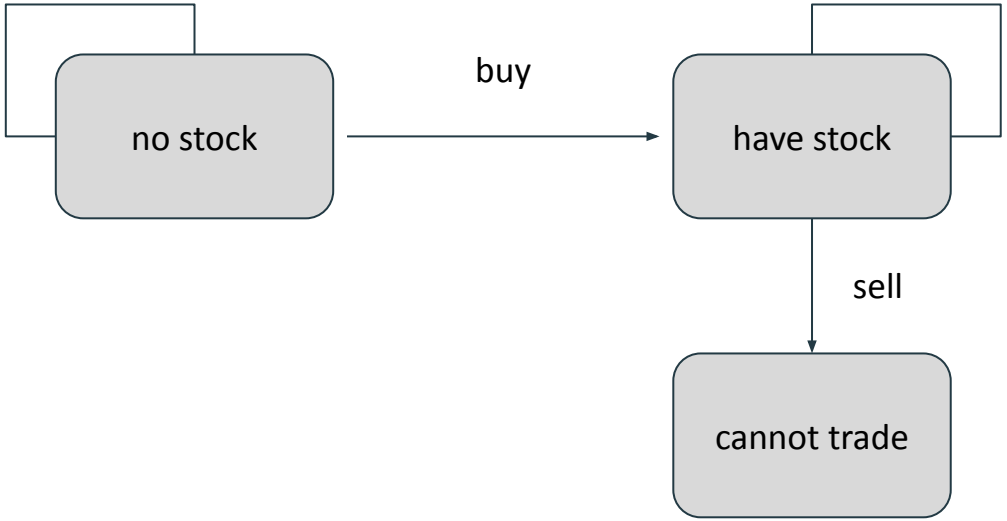
Longest Common Subsequence (LCS)

```
# Rolling DP reduce to 1D
if len(s1) < len(s2):
        s1, s2 = s2, s1

dp = [0 for j in range(len(s2)+1)]
for i in range(1, len(s1)+1):
        pre_dp = dp[:]
        for j in range(1, len(s2) + 1):
                if s1[i-1] == s2[j-1]:
                        dp[j] = pre_dp[j-1] + 1
                else:
                        dp[j] = max(pre_dp[j], dp[j-1])

return dp[len(s2)-1]
```

State Machine



max profit
profit = sell price - buy price

On Tree

Knapsack

Bitmask

```
@lru_cache(None)
def dp(used_mask, curr_idx):
    # termination condition
    if used_mask == ((1<<n) - 1):
        return

    curr_cost = compute_cost(idx)
    min_cost = math.inf
    for i in range(curr_idx+1, len(nums)):
        if used_mask & (1<<i) == 0: # not used
            suffix_cost = dp(used_mask | (1<<i), i)
            min_cost = min(min_cost, suffix_cost)
    return min_cost + curr_cost
```

Digit DP

Graph

Breath First Search (BFS)

```
visited = [False] * n
visited[src] = True
queue = deque([src]) # double-ended queue
while queue:
    node = queue.popleft()

    for neighbor in graph[node]:
        if not visited[neighbor]:
            visited[neighbor] = True
            queue.append(neighbor)
```

```
visited = [False] * n
visited[src] = True
level = [src]
while level:
    next_level = []
    for node in level:
        for neighbor in graph[node]:
            if not visited[neighbor]:
                visited[neighbor] = True
                next_level.append(neighbor)

    level = next_level
```

Depth First Search (DFS)

Union Find (Disjoint Set)

```
# Suppose node id is 0 to n-1
parent = [i for i in range(n)]
def find(i):
    if parent[i] != i:
        parent[i] = find(parent[i])
    return parent [i]

def union(i, j):
    # make root of j becomes a child of root of i
    parent[find(j)] = find(i)
```

```
# Example: find number of connected nodes
for a, b in edges:
    union(a, b)

unique_root = set()
for p in parent:
    unique_root.add(find(p))

return len(unique_root)
```

Union Find

```
class DSU:
    def __init__(self, size):
        self.parent = [i for i in range(size)]

    def find(self, i):
        if self.parent[i] != i:
            self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        self.parent[self.find(i)] = self.find(j)
```

Topological Sort (BFS a.k.a Kahn's)

```
graph = [[] for i in range(n)]
in_degree = [0 for i in range(n)]  # [0] * n
for u, v in edges:
    graph[u].append(v)
    in_degree[v] += 1
```

```
order = []
queue = deque([i for i in range(n) if in_degree[i] == 0])
while queue:
    curr_node = queue.popleft()
    order.append(curr_node)
    for next_node in graph[curr_node]:
        in_degree[next_node] -= 1
        if in_degree[next_node] == 0:
            queue.append(next_node)

len(order) == len(graph)  # cycle detection
```

Topological Sort (DFS)

```
graph = [[] for i in range(n)]
for u, v in edges:
        graph[u].append(v)

has_cycle = False
visited = [0] * n  # 0: not visit, 1: visiting, 2: visited
results = []
```

```
def dfs(node):
        nonlocal has_cycle
        if visited[node] == 2:
                return
        elif visited[node] == 1:
                has_cycle = True
                return

        visited[node] = 1
        for next_node in graph[node]:
                dfs(next_node)
        visited[node] = 2
        results.append(node)

for node in range(n):
        dfs(node)

return results[::-1] if has_cycle == False else []
```

Dijkstra

```
# Time complexity = O((V+E)logV), Space complexity = O(V+E)
# Requires non-negative weights
min_dist = [math.inf] * n
min_dist[src] = 0
heap = [(0, src)]
while heap:
    prev_dist, node = heapq.heappop(heap)
    if min_dist[node] < prev_dist:
        continue
    min_dist[node] = prev_dist
    for next_node, dist in graph[node]:
        if min_dist[next_node] > prev_dist + dist:
            min_dist[next_node] = prev_dist + dist
            heapq.heappush(heap, (prev_dist + dist, next_node))

min_dist # minimum distance of src to node = min_dist[node]
```

Dijkstra within k Steps

```
# Time complexity = O(kVlogV + E), Space complexity = O(V+E)
min_dist = dict()  # {(node, step): dist}
heap = [(0, src, 0)]
while heap:
        prev_dist, node, step = heapq.heappop(heap)
        if (node, step) in min_dist and min_dist[(node, step)] < prev_dist:
                continue

        min_dist[(node, step)] = prev_dist

        if step >= max_step:
                continue

        for next_node, dist in graph[node]:
                if (next_node, step+1) not in min_dist or min_dist[(next_node, step+1)] > prev_dist + dist:
                        min_dist[(next_node, step+1)] = prev_dist + dist
                        heapq.heappush(heap, (prev_dist + dist, next_node, step+1))

min_dist # minimum distance of src to node = min_dist[node]
```

Minimum Spanning Tree (Kruskal's)

```
# Union-find
# Time complexity = O(ElogE + α(V))
# Space complexity = O(V + E)
parent = [i for i in range(n)]
def find(i):
    if parent[i] != i:
        parent[i] = find(parent[i])
    return parent[i]

def union(i, j)
    root_i = find(i)
    root_j = find(j)
    if root_i == root_j:
        return False
    else:
        parent[find(j)] = find(i)
        return True
```

```
edges.sort(key=lambda edge: edge[2])   # sort by cost

edge_count = 0
min_cost = 0
for u, v, cost in edges:
    if not union(u, v):
        min_cost += cost
        edge_count += 1

    if edge_count == n - 1:  # connected n node with n-1 edges
        break

return min_cost if edge_count == n - 1 else -1
```

Minimum Spanning Tree (Prim's)

```
# Heap + BFS, Time complexity = O(E log V), Space complexity = O(V + E)
heap = [(0, 0)] # (cost, node)
min_cost = 0
visited = [False] * n
while heap:
    cost, node = heapq.heappop(heap)
    if visited[node]:  # visited means the node has already been added with the minimum cost
        continue

    visited[node] = True  # position different from regular BFS. A node can be visited multiple time (with different cost)
    min_cost += cost

    for next_node, next_cost in graph[node]:
        if not visited[next_node]:
            heapq.heappush(heap, (next_cost, next_node))

return min_cost if all(visited) else -1
```

Bellman-Ford

```
# Time complexity = O(V*E), Space complexity = O(V)
# Can handle negative weight and detect negative cycle
dist = [math.inf] * n
dist[src] = 0
for i in range(n-1):
        for u, v, w in edges:
                if dist[u] + w < dist[v]:
                        dist[v] = dist[u] + w

for u, v, w in edges:
        if dist[u] + w < dist[v]:
                return -1 # negative cycle happens

# dist[node] is the minimum distance from src to node
```

Binary Search

Binary search

```
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2
    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
```

Lower Bound / bisect.bisect_left

```
left, right = 0, len(nums)  # ! boundary is different from regular BS
while left < right:   # ! using  '<', not '<='
        mid = left + (right - left) // 2
        if nums[mid] < target:    # finally nums[left] >= target
                left = mid + 1
        else:
                right = mid

return left
```

Upper Bound / bisect.bisect_right

```
left, right = 0, len(nums)  # ! boundary is different from regular BS
while left < right:   # ! using  '<', not '<='
        mid = left + (right - left) // 2
        if nums[mid] <= target:  # finally nums[left] > target
                left = mid + 1
        else:
                right = mid

return left
```

Binary Search on Rotated Array

```
# values in nums are uniuqe
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2
    if nums[mid] == target:
        return mid

    # left side is sorted
    if nums[left] < nums[mid]:
        if nums[left] <= target < nums[mid]:
            right = mid - 1
        else:
            left = mid + 1
    else: # right side is sorted
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1
```

```
# values in nums are not uniuqe, worst case O(N)
left, right = 0, len(nums) - 1
while left <= right:
    mid = left + (right - left) // 2
    if nums[mid] == target:
        return mid

    while left < mid and nums[left] == nums[mid]:
        left += 1

    if nums[left] < nums[mid]: # left side is sorted
        if nums[left] <= target < nums[mid]:
            right = mid - 1
        else:
            left = mid + 1
    else:  # right side is sorted
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1
```

On Solution Spaces

With Dynamic Programming

Interval

Merge Interval

```
def merge(intervals):
    merge = []
    intervals.sort()
    for interval in intervals:
        if not merge or merge[-1][1] < interval[0]:
            merge.append(interval)
        else:
            merge[-1][1] = max(merge[-1][1], interval[1])
    return merge
```

Max non-overlapping Interval

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1]) # faster
    count = 0
    end = intervals[0][1]
    for interval in intervals[1:]:
        if end <= interval[0]:
            end = interval[1]
        else:
            count += 1
    return count
```

Number of Max Overlap Intervals

```
# Line sweep: O(N log N)
def maxOverlapIntervals(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()
    count, max_count = 0, 0
    for _, val in events:
        count += val
        max_count = max(count, max_count)

    return max_count
```

```
# Heap: O(N log N)
def maxOverlapIntervals(intervals):
    intervals.sort()
    heap = []
    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heappop(heap)

        heapq.heappush(heap, end)

    return len(heap)
```

Min number of intervals to cover time

```
def intervalCovering(intervals, time) -> int:
    intervals.sort()  # O(N log N)
    num_intervals = 0
    end = 0
    i = 0
    while end < time:
        max_end = end
        while i < len(intervals) and intervals[i][0] <= end:
            max_end = max(max_end, intervals[i][1])
            i += 1

        if max_end == end:
            return -1
        end = max_end
        num_intervals += 1

    return num_intervals
```

Sliding Window

Non-fixed Size Sliding Window

```
# find number of subarrays sum up smaller then target
left = 0
total = 0
count = 0
for right in range(len(nums)):
    # expand
    total += nums[right]

    # shrink
    while total > target:
        total -= nums[left]
        left += 1

    # update
        count += (right + 1 - left)
```

Binary Tree

Breath First Search (BFS)

```
# Just traverse
queue = deque([root])
while queue:
    node = queue.popleft()
    print(node.val)

    if node.left:
        queue.append(node.left)
    if node.right:
        queue.append(node.right)
```

```
# get the level structures
levels = []
level = [root]
while level:
    next_level = []
    for node in level:
        if node.left:
            next_level.append(node)
        if node.right:
            next_level.append(node)
    level = next_level
    levels.append(level)

print(levels)
```

Depth First Search (DFS)

For binary Tree, DFS only has three templates

- Inorder traversal: left -> current -> right
- Preorder traversal: current -> left -> right
- Postorder traversal: left -> right -> current

DFS in binary tree does not require a visited set or list because binary tree is a directed acyclic graph (DAG).

A visited set or list is required if cycles might exist.

Inorder (DFS branch)

```
def inorder(node):
    if not node:
        return
    inorder(node.left)
    print(node.val)
    inorder(node.right)
```

```
# using iterative (not recommend)
def inorder(node):
    if not node:
        return
    stack = []
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        print(node.val)
        node = node.right
```

Postorder (DFS branch)

```
def postorder(node):
    if not node:
        return
    postorder(node.left)
    postorder(node.right)
    print(node.val)
```

```
# using iterative (not recommend)
def postorder(node):
    if not node:
        return
    stack1 = [root]
    stack2 = []
    while stack1:
        node = stack1.pop()
        stack2.append(node)
        if node.left:
            stack1.append(node.left)
        if node.right:
            stack2.append(node.right)
    while stack2:
        print(stack2.pop().val)
```

Preorder (DFS branch)

```
def preorder(node):
    if not node:
        return
    print(node.val)
    preorder(node.left)
    preorder(node.right)
```

```
# using iterative (not recommend)
def preorder(node):
    if not node:
        return
    stack = [node]
    while stack:
        node = stack.pop()
        print(node.val)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
```

Preorder constructs BST

```
def dfs(preorder, left, right) -> Optional[TreeNode]:
        if left == right:
                return None

        node = TreeNode(preorder[left])
        gt_idx = bisect.bisect_right(preorder,
node.val, lo=left+1)
        node.left = dfs(preorder, left + 1, gt_idx)
        node.right = dfs(preorder, gt_idx, right)
        return node

return dfs(preorder, 0, len(preorder)
```

```
# Explanation
# preorder: current -> left -> right
# because all values in preorder are uqniue, use
binary search upper bound to find gt_idx
# current = preorder[0]
# left_part = preorder[0+1: gt_idx]
# right_part = preorder[gt_idx: ]
```

Inorder and Preorder construct Tree

Inorder and Postorder construct Tree

Preorder and Postorder construct Tree

Lowest Common Ancestor (LCA)

```
def lowestCommonAncestor(node, p, q):
    if not node or node == p or node == q:
        return node

    left = lowestCommonAncestor(node.left, p, q)
    right = lowestCommonAncestor(node.right, p, q)

    if left and right:
        return node
    return left if left else right
```

```
def lcaDeepestLeaves(node): # return depth, node
    if not node:
        return 0, None

    left = lcaDeepestLeaves(node.left)
    right = lcaDeepestLeaves(node.right)

    if left[0] > right[0]:
        return left[0] + 1, left[1]
    elif left[0] < right[0]:
        return right[0] + 1, right[1]
    else:
        return left[0] + 1, node

return lcaDeepestLeaves(root)[1]
```

Path Sum

```
# Check at least one path(root, leaf) can sum up to target
def dfs(node, target):
    if not node:
        return False
    if not node.left and not node.right:
        return node.val == target

    left = dfs(node.left, target-node.val)
    right = dfs(node.right, target-node.val)
    return left or right

dfs(root, target)
```

```
# find all paths(root, leaf) sum up to target
paths = []
def dfs(node, prev_path, target):  # backtracking DFS
    if not node:
        return
    if not node.left and not node.right:
        if node.val == target:
            paths.append(prev_path + [node.val])
        return

    prev_path.append(node.val)
    dfs(node.left, prev_path, target - node.val)
    dfs(node.right, prev_path, target - node.val)
    prev_path.pop()

dfs(root, [], target)
return paths
```

Path Sum with Prefix Sum

```
# Path can be any node downward to another node
# using the concept of subarray sum up to target
count = 0
prefix_sum_freq = defaultdict(int)
def dfs(node, prev_sum, target):
    nonlocal count
    if not node:
        return
    curr_sum = prev_sum + node.val
    if curr_sum == target:
        count += 1
    count += prefix_sum_freq[curr_sum-target]
    prefix_sum_freq[curr_sum] += 1
    dfs(node.left, curr_sum, target)
    dfs(node.right, curr_sum, target)
    prefix_sum_freq[curr_sum] -= 1

dfs(root, 0, target)
return count
```

```
# subarray sum up to target
def subarray_sum(nums, target):
    count = 0
    curr_sum = 0
    prefix_sum_freq = defaultdict(int)
    for num in nums:
        curr_sum += num
        if curr_sum == target:
            count += 1

        count += prefix_sum_freq[curr_sum-target]
        prefix_sum_freq[curr_sum] += 1

    return count
```

Max Path Sum

```
# Path can be arbitrary
max_sum = -math.inf
def dfs(node) -> int: # return max_path_sum involve node and at most one child's path
    nonlocal max_sum
    if not node:
        return 0

    left = max(dfs(node.left), 0)
    right = max(dfs(node.right), 0)

    max_sum = max(max_sum, node.val + left + right)
    return node.val + max(left, right)

dfs(root)
return max_sum
```

Priority Queue (Heap)

Prefix Sum

Subarray Sum to Target

```
# subarray sum up to target
def subarray_sum(nums, target):
    count = 0
    curr_sum = 0
    prefix_sum_freq = defaultdict(int)
    for num in nums:
        curr_sum += num
        if curr_sum == target:
            count += 1

        count += prefix_sum_freq[target - curr_sum]
        prefix_sum_freq[curr_sum] += 1

    return count
```

Quick Select

Two part (Unique value)

```
def partition(left, right):
    random_idx = random.randint(left, right)
    swap(random_idx, right)  # move to rightmost

    pivot_value = nums[right]
    pivot_idx = left
    for i in range(left, right):
        if nums[i] < pivot_value:
            swap(i, pivot_idx)
            pivot_idx += 1

    swap(pivot_idx, right)
    return pivot_idx  # pivot_value = nums[pivot_idx]
```

```
# Example, find k smallest value
target = k - 1
left, right = 0, len(nums) - 1
while left <= right:
    pivot_idx = partition(left, right)
    if pivot_idx == target:
        return nums[pivot_idx]
    elif pivot_idx < target:
        left = pivot_idx + 1
    else:
        right = pivot_idx - 1
```

Worst case is O(N^2) if a value duplicates extremely large times

Three part (Duplicate value)

```
def partition(left, right):
    random_idx = random.randint(left, right)
    swap(random_idx, right)  # move to rightmost

    pivot_value = nums[right]
    lt_idx, i, gt_idx = left, left, right
    while i <= right_idx:
        if nums[i] < pivot_value:
            swap(i, lt_idx)
            i, lt_idx = i+1, lt_idx + 1
        elif nums[i] > pivot_value:
            swap(i, gt_idx)
            gt_idx -= 1
        else:
            i += 1

    swap(i, right)
    return lt_idx, gt_idx
```

```
# Example, find k smallest value
target = k - 1
left, right = 0, len(nums) - 1
while left <= right:
    lt_idx, gt_idx = partition(left, right)
    if lt_idx > target:
        right = lt_idx - 1
    elif gt_idx < target:
        left = gt_idx + 1
    else:
        return nums[lt_idx]
```

Better performance for dealing with duplicate value

Backtracking

Linked List

Traverse / Delete

```
def traverse(head):
    node = head
    while node:
        node = node.next
```

```
def delete_node(head, val):
    dummy = ListNode(0, next=head)
    node = head
    prev = dummy
    while node:
        if node.val == val:
            prev.next = node.next
            node = node.next
        else:
            prev, node = prev.next, node.next

    return dummy.next
```

Reverse

```
def reverse(head):
    prev = None
    while head:
        next_node = head.next
        head.next = prev
        prev = head
        head = next_node
    return head
```

```
# very tricky, need to draw plot to understand
def reverse_between(head, m, n):
    dummy = ListNode(0, next=head)
    prev = dummy

    # find reversed start
    for _ in range(m-1):
        prev = prev.next

    curr = prev.next # curr is the start point
    for _ in range(n - m):
        next_node = curr.next
        curr.next = next_node.next
        next_node.next = prev.next
        prev.next = next_node

    return dummy.next
```

Merge two sorted Link lists

```
def merge(head1, head2):
    dummy = ListNode()
    node = dummy
    while head1 and head2:
        if head1.val < head2.val:
            node.next = head1
            head1 = head1.next
        else:
            node.next = head2
            head2 = head2.next
        node = node.next

    node.next = head1 if head1 else head2
    return dummy.next
```

Slow and Fast

```
# O(N)
def find_middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

```
def find_k_node_from_back(head, k):
    slow = fast = head
    for _ in range(k):
        if not fast:
            return None
        fast = fast.next

    while fast:
        slow = slow.next
        fast = fast.next

    return slow
```

Hare and Tortoise

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

```
# Math problem
def detectCycleStart(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            break

    if slow != fast: # no cycle
        return None

    slow = head
    while slow != fast:
        slow = slow.next
        fast = fast.next

    return slow
```

First meet: Slow = x + y, fast = x + y + kc = 2 (x+y) => x = kc - y

fast go (kc - y) step will reach x, slow go x step from start will reach x

Palindrome

Palindrome

```
def is_palindrome(s) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

```
# TC = O(N^2), SC = O(1)
def longest_substring_palindrome_len(s) -> str:
    def expand_from_center(left, right):   # O(N) for one time
        while left >= 0 and right < len(s):
            if s[left] != s[right]:
                break
            left, right = left-1, right+1
        return left + 1, right - 1

    max_len = 0
    for i in range(len(s)):
        left_odd, right_odd = expand_from_center(i, i)
        max_len = max(max_len, right_odd - left_odd + 1)
        if i + 1 < len(s):
            left_even, right_even = expand_from_center(i, i+1)
            max_len = max(max_len, right_even - left_even + 1)

    return max_len
```

Palindrome with k operations

```
def longest_subsequence_palindrome_len(s) -> str:
    # LCS with O(N^2) Time and O(N) Space
    dp = [0 for i in range(len(s)+1)]
    reversed_s = s[::-1]
    for i in range(1, len(s)+1):
        pre_dp = dp[:]
        for j in range(1, len(s)+1):
            if s[i-1] == reversed_s[j-1]:
                dp[j] = pre_dp[j]
            else:
                dp[j] = max(dp[j-1], pre_dp[j])

    return dp[len(s)]
```

Prime

Prime number

```
def is_prime(n) -> bool:
    if n == 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
# Sieve of Eratosthenes: find all primes within n
def sieve(n) -> List[int]:
    is_prime = [True] * (n+1)
    is_prime[0:2] = [False, False]
    for i in range(2, int(n ** 0.5) + 1):
        if is_prime[i]:
            for j in range(i*i, n+1, i):
                is_prime[i] = False
    return [i for i, val in enumerate(is_prime) if val]
```

Prime factors

```
# O(sqrt(n))
def prime_factors(n) -> List[int]:
    factors = []
    i = 2
    while i * i <= n:
        while n % i == 0:
            factors.append(i)
            n //= i
        i += 1
    if n > 0:
        factors.append(i)
    return factors
```

```
def first_n_primes(n):
    primes = []
    i = 2
    while len(primes) < n:
        if is_prime(i):
            primes.append(i)
        i += 1
    return primes
```

GCD / LCM

GCD / LCM

```
def gcd(a, b) -> int:
    return a if b == 0 else gcd(b, a % b)
```

```
def lcm(a, b) -> int:
    return a * b / gcd(a, b)
```

$$\mathrm{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Permutation / Combination

Permutation

```
# unique
results = []
visited = [False] * len(nums)
def backtrack(path):
    if len(path) == len(nums):
        results.append(path[:])
        return
    for i in range(len(nums)):
        if not visited[i]:
            visited[i] = True
            path.append(i)
            backtrack(path)
            path.pop()
            visited[i] = False
backtrack([])
return results
```

```
# duplicate
nums.sort()
results = []
visited = [False] * len(nums)
def backtrack(path):
    if len(path) == len(nums):
        results.append(path[:])
        return
    for i in range(len(nums)):
        if visited[i]:
            continue
        if i > 0 and nums[i-1] == nums[i] and not visited[i-1]:
            continue
        visited[i] = True
        path.append(i)
        backtrack(path)
        path.pop()
        visited[i] = False
backtrack([])
return results
```

Combination

```
# input is unique value
def combine(n, k):
    results = []
    def backtrack(start, prefix):
        if len(prefix) == k:
            results.append(prefix[:])
            return
        for i in range(start, n+1):
            prefix.append(i)
            backtrack(start+1, prefix)
            prefix.pop()
        return
    backtrack(1, [])
    return results

combine(4, 2)
```

```
# input has duplicate / output cannot duplicate
def combine(nums, k):
    results = []
    nums.sort()
    def backtrack(start, prefix):
        if len(prefix) == k:
            results.append(prefix[:])
            return
        for i in range(start, len(nums)):
            if nums[i] == prefix[-1]:
                continue
            prefix.append(i)
            backtrack(start+1, prefix)
            prefix.pop()
        return
    backtrack(0, [])
    return results
combine([1, 1, 2, 4], 2) # [[1, 2], [1, 4], [2, 4]]
```

Combination

```
# input has duplicate / output can have duplicate
def combine(nums, k):
    results = []
    nums.sort()
    def backtrack(start, prefix):
        if len(prefix) == k:
            results.append(prefix[:])
            return
        for i in range(start, len(nums)):
            prefix.append(i)
            backtrack(start, prefix)
            prefix.pop()
        return
    backtrack(0, [])
    return results
combine([1, 2, 4], 2) # [[1, 1], [1, 2], [1, 4], [2, 2], [2, 4], [4, 4]]
```

Monotonic Stack

Monotonic Stack

```
stack = []
for num in nums:
    while stack and stack[-1] > num:
        stack.pop()
    stack.append(num)
```

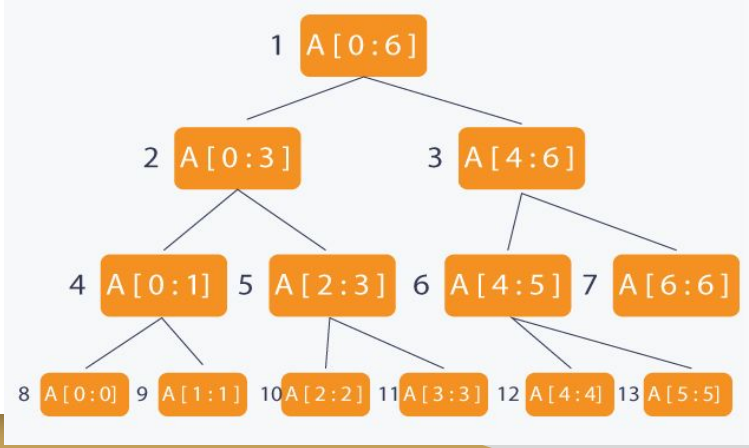
Segment Tree

```python
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (4*self.n)
        self._build(data, 1, 0, self.n-1)

    def _build(self, data, node, left, right):
        if left == right:
            self.tree[node] = data[left]
        else:
            mid = left + (right - left) // 2
            self._build(data, node*2, left, mid)
            self._build(data, node*2+1, mid+1, right)
            self.tree[node] = self.tree[node*2] + self.tree[node*2+1]

    def query(self, left, right):
        return self._query(1, 0, self.n-1, left, right)

    def _query(self, node, l, r, left, right):
        if right < l or left > r:
            return 0
        if left <= l and r <= right:
            return self.tree[node]

        mid = (l + r) // 2
        return (self._query(node*2, l, mid, left, right) +
            self._query(node*2 + 1, mid+1, r, left, right))
```

```python
    def update(self, idx, value):
        self._update(1, 0, self.n-1, idx, value)

    def _update(self, node, l, r, idx, value):
        if l == r:
            self.tree[node] = value
        else:
            mid = (l + r) // 2
            if idx <= mid:
                self._update(node*2, l, mid, idx, value)
            else:
                self._update(node*2+1, mid+1, r, idx, value)
            self.tree[node] = self.tree[node*2] + self.tree[node*2+1]
```